

PRACTICAL WEEK – 10

Experiment 1: Inode Structures, Hard Links and Symbolic Links

Aim

To investigate **inode structures** using Linux commands such as `ls -i`, `stat`, and `find`, and to create **hard links** and **symbolic links** to understand their impact on inode allocation.

Theory

An **inode (index node)** is a data structure used by a Linux/Unix file system to store important information about a file. It contains metadata such as the file type, permissions, owner, group, file size, timestamps, and pointers to the blocks containing the actual file data. The inode does not normally store the file name; the directory stores the name and maps it to an inode number.

The following commands are commonly used to examine inode information:

- `ls -i` – displays the **inode number** of a file.
- `stat` – displays detailed information about a file, including its inode number, permissions, size, and timestamps.
- `find` – can be used to search for files and locate files associated with particular inode numbers.

A **hard link** is another directory entry that points to the **same inode** as the original file. Therefore, the original file and its hard link share the same data and inode. Deleting one hard link does not delete the actual data as long as another hard link to the inode exists.

A **symbolic link (soft link)** is a separate file that stores the path of another file. It has its **own inode number** and points to the target file. If the original file is deleted or moved, the symbolic link may become a **broken link**.

Thus, hard links share the same inode, while symbolic links have a separate inode and refer to the original file through its path.

Program / Commands

```
echo "Hello Linux" > original.txt

ls -li original.txt

stat original.txt

ln original.txt hardlink.txt

ln -s original.txt symlink.txt

ls -li original.txt hardlink.txt symlink.txt

stat hardlink.txt

stat symlink.txt

find . -inum <inode_number>
```

Output:

```

chakrika@Chakrika:~$ mkdir week10
chakrika@Chakrika:~$ cd week10
chakrika@Chakrika:~/week10$ echo "Hello Linux" > original.txt
chakrika@Chakrika:~/week10$ ls
original.txt
chakrika@Chakrika:~/week10$ ls -li original.txt
46155 original.txt
chakrika@Chakrika:~/week10$ stat original.txt
  File: original.txt
  Size: 12          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 46155       Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/chakrika)   Gid: ( 1000/chakrika)
Access: 2026-09-28 06:36:21.884046674 +0000
Modify: 2026-09-28 06:36:21.884046674 +0000
Change: 2026-09-28 06:36:21.884046674 +0000
 Birth: 2026-09-28 06:36:21.884046674 +0000
chakrika@Chakrika:~/week10$ ln original.txt hardlink.txt
chakrika@Chakrika:~/week10$ ln -s original.txt symlink.txt
chakrika@Chakrika:~/week10$ ls -li original.txt hardlink.txt symlink.txt
46155 -rw-r--r-- 2 chakrika chakrika 12 Sep 28 06:36 hardlink.txt
46155 -rw-r--r-- 2 chakrika chakrika 12 Sep 28 06:36 original.txt
14462 lrwxrwxrwx 1 chakrika chakrika 12 Sep 28 06:36 symlink.txt -> original.txt
chakrika@Chakrika:~/week10$ find . -inum 123456
chakrika@Chakrika:~/week10$ find . -inum 46155
./original.txt
./hardlink.txt
chakrika@Chakrika:~/week10$ |

```

Experiment 2: File Reading and Writing Using mmap()

Aim

To design a program using the **mmap()** system call for file reading and writing and compare memory-mapped I/O with traditional read() and write() operations in terms of implementation and efficiency.

Theory

Memory mapping (mmap()) is a system call provided by Unix/Linux operating systems that maps a file or a portion of a file directly into the process's virtual memory. Once a file is mapped, the program can access the file contents using normal memory operations such as reading and modifying an array.

The `mmap()` system call reduces the need for repeated explicit `read()` and `write()` system calls. The operating system manages the movement of data between memory and storage using virtual memory and paging mechanisms.

Important functions used in the program are:

- `open()` – opens a file and returns a file descriptor.
- `mmap()` – maps the file into the process's memory space.
- `munmap()` – removes the mapping from the process's address space.
- `msync()` – synchronizes modified memory contents with the file.
- `close()` – closes the file descriptor.

Traditional file I/O uses functions such as `read()` and `write()` to explicitly transfer data between the file and a buffer. In contrast, `mmap()` allows the program to treat the mapped file region like an array in memory.

Memory-mapped I/O can be useful when working with large files or when the same file data needs to be accessed repeatedly. However, it requires careful handling of memory mapping and synchronization.

Program

```
C mmap_file.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <fcntl.h>
4  #include <sys/mman.h>
5  #include <sys/stat.h>
6  #include <unistd.h>
7  #include <string.h>
8
9  int main()
10 {
11     int fd;
12     struct stat sb;
13     char *mapped;
14
15     fd = open("mmapfile.txt", O_RDWR | O_CREAT, 0666);
16
17     if (fd == -1)
18     {
19         perror("open");
20         return 1;
21     }
22
23     const char *text = "Hello from mmap!";
24
25     write(fd, text, strlen(text));
26
27     fstat(fd, &sb);
28
29     mapped = mmap(NULL, sb.st_size,
30                  PROT_READ | PROT_WRITE,
31                  MAP_SHARED, fd, 0);
32
33     if (mapped == MAP_FAILED)
34     {
35         perror("mmap");
36         close(fd);
37         return 1;
38     }
39
40     printf("File content using mmap(): %s\n", mapped);
41
42     munmap(mapped, sb.st_size);
43     close(fd);
44
45     return 0;
46 }
```

OUTPUT:

```
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ cd "/mnt/c/Users/panak/Downloads/OSSP PRACTICALS"
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ls
counter.c  counter_mutex.c  deadlock_prevention.c  inode_commands.txt  original.txt  producer_consumer.exe
counter.exe  counter_mutex.exe  deadlock_prevention.exe  mmap_file.c  producer_consumer.c
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc mmap_file.c -o mmap_file
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./mmap_file
File content using mmap(): Hello from mmap!
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$
```

Result:

The file was successfully accessed using the `mmap()` system call. Memory-mapped I/O allows a file to be accessed directly through memory, reducing the need for repeated explicit `read()` and `write()` operations. Compared with traditional file I/O, `mmap()` can provide efficient access to file data, although it requires additional handling of memory mapping and synchronization.